Team Name : RedRob

Team Leader Name : Rupin Ajay

Problem Statement : Rank 100K+ candidates for a Senior AI Engineer role using config-driven multi-view matching under CPU-only constraints (300s, 16GB)

Solution Overview

A config-driven candidate ranking engine that scores profiles against a Senior AI Engineer JD using: multi-view TF-IDF semantic similarity (4 views: full profile, skills, title+headline, character n-grams), skill evidence scoring with description cross-validation, career trajectory quality (promotions, stability, switcher detection), soft title gates that let evidence override hard caps, and behavioral signal adjustment — all packaged in a single 30s CPU-only pipeline with zero external API calls.

Key differentiators:

- Config-driven architecture — swap config.json for a different role with zero code changes
- Soft gates instead of hard title caps — career switchers with strong evidence can score up to their tier cap
- Multi-view TF-IDF (incl. char n-grams) catches sub-word patterns and acronyms that simple keyword matching misses
- Description cross-validation rewards skills that appear in actual career descriptions (real usage vs keyword stuffing)
- Fully deterministic — same input always produces identical output, seeded per-candidate for reproducible reasoning

JD Understanding & Candidate Evaluation

Key JD Requirements Extracted:

• Senior AI Engineer — advanced ML/DL skills, production experience, vector databases, eval frameworks
• Archetype tiers: S (AI Research Scientist/Engineer), A (ML Engineer), B (Data Scientist), C (SWE/Analyst), D (Non-Engineering)
• Skill groups: Embedding & Representation Learning, Vector DB & Search, Production ML, Evaluation & Benchmarking
• Career signals: promotions within same company, role stability (12-48 months), AI career trajectory
• Education: MS/PhD in quantitative fields, research degree bonus
• Location: India-based preference with tiered scoring

Candidate Signals Used:

• Title archetype match → configurable tier caps with soft evidence gates
• Skill proficiency × duration × endorsements → group scores blended by config weights
• Career history → promotions, job stability, career switcher bonus
• TF-IDF semantic similarity → full profile + skills + title/headline + char n-grams (4-view blend)
• Education → degree level × field relevance, research bonus
• Behavioral → 6 signals (github, publications, patents, etc.) with narrowed clamp [0.55, 1.15]
• Honeypot detection → 8 rule patterns for suspicious profiles

Weights are all config-driven and sum to 1.0, enabling easy role-specific tuning.

Ranking Methodology

Retrieval & Scoring Pipeline:

1. Load config.json → all weights, thresholds, archetypes, skill groups, honeypot rules
2. Honeypot detection → flag and exclude suspicious profiles (8 rule patterns)
3. Archetype scoring → title match against hierarchical tiers with soft evidence gates
4. Skill evidence scoring → per-group proficiency×duration×endorsement with description validation +0.30 trust boost
5. Career trajectory → promotions score, stability band, switcher bonus +0.15
6. Multi-view TF-IDF → 4 views blended: full_profile(40%) + skills(25%) + title_headline(15%) + char_ngram(20%)
7. Education → degree+field score + research degree bonus +0.10
8. Behavioral modulation → 6 signals, clamp [0.55, 1.15]
9. Soft gate → tier_cap = base + (max - base) × evidence_factor for D-tier transitions
10. Sort by score desc, tie-break by candidate_id asc → top 100 with unique reasoning

Scoring Formula:

raw = W_arch×A + W_skills×B + W_career×C + W_loc×D + W_edu×E + W_semantic×F

final = clamp(raw × behavioral_mod, 0, 1), then soft-gate applied per tier

All W values sum to 1.0 in config.

Explainability & Data Validation

Explainability:
• Every candidate gets a unique reasoning string generated from 12 intro templates × 21 conclusion variations, seeded by candidate_id for reproducibility
• Reasoning includes: matched archetype tier, top skill groups, career trajectory highlights, location fit, education tier, and behavioral adjustment
• No LLM used — all reasoning is rule-based and deterministic, so explanations are always grounded in actual candidate data

Hallucination Prevention:
• All reasoning text is template-driven with variables filled from computed scores — zero generative text
• Skill descriptions are cross-validated: a named skill gets +0.30 trust boost only if it appears in career descriptions
• Soft gates cap scores by evidence, preventing over-scoring based on title alone
• Honeypot detector flags 8 categories of suspicious profiles (template responses, missing fields, unlikely skills, etc.)

Handling Low-Quality Data:
• Missing fields get minimal default scores (e.g., default_proficiency: beginner = 0.25)
• Career gap detection identifies roles with 0-month durations as unreliable
• Behavioral clamp narrowed to [0.55, 1.15] reduces outlier impact from noisy signals
• Honeypots are excluded from the ranked pool entirely

End-to-End Workflow

1. candidates.jsonl (100K profiles, ~280MB) loaded line-by-line
2. Config-driven initialization → load all weights, tiers, groups from config.json
3. Honeypot filter → scan each profile against 8 rule patterns, exclude flagged
4. Multi-View TF-IDF (4 views) → fit vectorizer on JD+profiles, compute cosine similarity per view, blend with config weights (~20s)
5. Candidate scoring loop → for each of 100K candidates: archetype → skills → career → location → education → semantic → behavioral → soft gate
6. Sort → desc by score, asc by candidate_id for ties
7. Top 100 → generate unique reasoning per candidate (seeded templates)
8. Output → submission.csv (rank, candidate_id, score, reasoning), validate via official validator
9. Gradio UI (optional) → upload file, see live progress bar, download CSV

Total runtime: 30.2s on 100K candidates (10x under 300s limit). Memory: <1GB.

System Architecture

Layered Architecture (single-file design for hackathon constraints):

rank.py (core engine):

load_config() - parse and validate config.json
compute_tfidf_scores() - 4-view TF-IDF with sklearn TfidfVectorizer
score_archetype() - tier matching + soft gate
score_skills_evidence() - per-group scoring + description validation
score_career_pattern() - promotions, stability, switcher detection
get_behavioral_mod() - 6-signal adjustment
detect_honeypots() - 8 rule-based pattern matchers
rank_candidates() - full pipeline orchestrator
gen_reasoning() - 100 unique template-based explanations
main() - CLI entry point

config.json (configuration layer):

weights (archetype, skills, career, location, education, semantic)
archetype tiers (S/A/B/C/D with caps and title patterns)
skill groups with proficiency scores and overrides
behavioral signals (6 signals with weights and thresholds)
honeypot rules (8 rule patterns)
multi-view semantic config (4 views, n-gram ranges, weights)

app.py (Gradio UI): upload, rank, download with live progress bar
validate_submission.py: official challenge validator

Results & Performance

Ranking Quality:
- 100 unique scores, 100 unique reasonings (seeded templates ensure variety)
- Score range: 0.7323 - 0.8753 (compressed due to narrowed behavioral clamp and evidence-gated soft caps)
- Top titles in top 100: AI Research Engineer (17), Recommendation Systems Engineer (16), ML Engineer (10)
- Non-engineering titles in top 100: 0 (all top candidates are technical/AI roles)
- Honeypots in top 100: 0 (43 total honeypots detected, excluded from pool)

Performance (100K candidates):
- Total runtime: 30.2s (300s limit: 10x headroom)
- Memory: <1GB RAM (16GB limit: 16x headroom)
- CPU-only, no GPU, no network calls

TF-IDF Breakdown (4 views):

full_profile: 12.6s (mean sim 0.028) — catches overall JD alignment
skills: 0.8s (mean sim 0.022) — skill name semantic matching
title_headline: 0.4s (mean sim 0.005) — role title alignment
char_ngram: 8.2s (mean sim 0.107) — sub-word patterns, highest mean similarity

Config weights: archetype 0.35, skills 0.25, career 0.12, location 0.05, education 0.08, semantic 0.15

Technologies Used

Core Stack:
• Python 3.11+ — primary language for data processing, scoring logic, and CLI
• scikit-learn TfidfVectorizer — multi-view TF-IDF with character n-gram support for sub-word pattern matching
• numpy — vectorized operations for score blending and matrix math
• Gradio 6.19 — interactive web UI with file upload, live progress bars (gr.Progress + tqdm), and CSV download

Why these choices:
• No external API calls allowed (no network) — TF-IDF runs entirely in-process
• No GPU available — sklearn TfidfVectorizer is CPU-optimized and fast
• Gradio chosen over Flask/Streamlit for native HuggingFace Spaces integration and built-in file handling
• Config-driven architecture (JSON over code) enables role swapping without redeployment

Deployment:
• HuggingFace Spaces — free CPU-only hosting with Gradio SDK (6.19)
• GitHub — private repo (made public for submission) with full version history

Submission Assets

GitHub Repository:
https://github.com/rupinajay/redrob-hackathon

HuggingFace Space (interactive UI):
https://rupinajay-redrob-ranker.hf.space

Key Files:
• rank.py — config-driven ranking engine
• config.json — all weights, thresholds, and parameters
• app.py — Gradio web UI
• validate_submission.py — official validator
• requirements.txt — scikit-learn, numpy, gradio
• submission_metadata.yaml — team info, methodology

Team:
• Rupin Ajay (SDE) — ranking engine, config system, Gradio UI, deployment
• Sanjay PN (ML Engineer) — TF-IDF multi-view design, skill scoring, career trajectory signals

redrob | H2S HACK2SKILL

INDIA.RUNS

Thank You!

Build what next India runs on

Rupin Ajay · Sanjay PN · Redrob Hackathon 2026

THANK YOU